Java Backend Interview Series

Chapter 1: Core Java Foundations

Sub-Chapter 1.4: Collections - Map Implementations

HashMap Internals | equals() & hashCode() | All Map Implementations

52 Interview Questions | Real-Life Analogies | Code Examples | Cheat Sheet

1. The Map Interface — Overview

A **Map** stores key-value pairs where each key is unique. Unlike Collection, Map is a separate hierarchy: it does not extend Iterable or Collection. The core interface is java.util.Map<K,V> with sub-interfaces SortedMap, NavigableMap, and ConcurrentMap. The six main implementations — HashMap, LinkedHashMap, TreeMap, ConcurrentHashMap, EnumMap, WeakHashMap — differ in ordering, thread-safety, and performance.

Implementation	Order	Thread-Safe	Null Key	Null Value	Complexity
HashMap	None (random)	No	1 allowed	Yes	O(1) avg get/put
LinkedHashMap	Insertion or access	No	1 allowed	Yes	O(1) get/put
TreeMap	Sorted by key	No	No	Yes	O(log n) get/put
ConcurrentHashMap	None	Yes (CAS)	No	No	O(1) avg, low contention
EnumMap	Enum ordinal	No	No	Yes	O(1), array-backed
WeakHashMap	None	No	1 (weak ref)	Yes	O(1); keys GC-able

2. HashMap Internals — The Library Analogy

Before diving into code, let's understand HashMap with a **real-life analogy**. Imagine a **large library** that stores books. Instead of searching every shelf for a book, the librarian uses a clever system to find any book in near-constant time.

The Library = HashMap

Books are stored on numbered shelves (0 to 15). When you bring a book to store, the librarian reads its ISBN (the KEY), runs a quick calculation on it (the HASH FUNCTION), and that calculation tells exactly which shelf number (BUCKET INDEX) to use. To retrieve the book later: give the ISBN, run the same calculation, go directly to that shelf. No need to search the entire library!

The mapping in detail:

Real-World	HashMap Concept	Java Term
Library building	The entire storage structure	HashMap object
Numbered shelves (0-15)	Fixed-size array of buckets	Node[] table (default 16 slots)
ISBN of a book	The key's unique identifier	K key
Librarian's calculation on ISBN	Converting key to shelf number	hashCode() + index formula
Shelf number result	Which array slot to use	bucket index = hash & (n-1)
Books stacked on same shelf	Multiple keys mapping to same slot	Linked list / Tree (collision)
Book content	The value stored	V value
ISBN card in the book	Key stored alongside value	Entry node: key, value, hash, next
Library expansion (add more shelves)	Growing the array	resize() / rehash at 75% capacity

Concrete real-life example — storing Employee objects by ID:

```
// Real-life scenario: HR system stores Employee objects, looked up by employeeId
// Key = String employeeId ("EMP-001"), Value = Employee object
HashMap<String, Employee> hrSystem = new HashMap<>();
// STORING: "Where does EMP-001 go?"
// Step 1: Java calls "EMP-001".hashCode()
// hashCode() of "EMP-001" might return, say, 1234567
// Step 2: HashMap spreads the hash: h = 1234567 ^ (1234567 >>> 16) = ~1234499
// Step 3: Find the bucket: index = 1234499 & (16-1) = 1234499 & 15 = 3
// (bitwise AND with 15 = last 4 bits = value between 0 and 15)
// Step 4: Store Employee at table[3]
hrSystem.put("EMP-001", new Employee("Alice", "Engineering"));
hrSystem.put("EMP-002", new Employee("Bob", "Marketing"));
hrSystem.put("EMP-003", new Employee("Carol", "Engineering"));
// RETRIEVING: "Find EMP-001"
// Step 1: hash("EMP-001") = 1234499 (same calculation)
// Step 2: index = 1234499 & 15 = 3 (same bucket)
// Step 3: Go to table[3], check node.key.equals("EMP-001") -> true -> return Employee
Employee alice = hrSystem.get("EMP-001"); // O(1) — no searching!
```

3. HashMap — Deep Dive: Array + Buckets + Tree

HashMap is backed by an **array of Node objects** called table. Each array slot is a **bucket**. When two keys hash to the same bucket (a **collision**), their nodes are chained. Java 8+ upgrades a bucket from a **linked list to a red-black tree** when it has 8 or more entries — improving worst-case lookup from O(n) to O(log n).

Internal Node structure:

```
// The actual HashMap.Node class (simplified from JDK source)
static class Node<K,V> implements Map.Entry<K,V> {
final int hash; // cached hash of the key (avoids recomputing on resize)
final K key; // the key (e.g. "EMP-001")
V value; // the value (e.g. Employee object)
Node<K,V> next; // pointer to next node in this bucket (for chaining)
// Constructor, getKey(), getValue(), setValue(), equals(), hashCode()...
}
// TreeNode (used when bucket size >= 8) — extends LinkedHashMap.Entry
static final class TreeNode<K,V> extends LinkedHashMap.Entry<K,V> {
TreeNode<K,V> parent, left, right, prev;
boolean red;
// Red-black tree node: O(log n) lookup instead of O(n) linked list scan
}
// The HashMap itself
public class HashMap<K,V> {
transient Node<K,V>[] table; // THE ARRAY OF BUCKETS
transient int size; // number of key-value pairs
int threshold; // resize when size > threshold (capacity * loadFactor)
final float loadFactor; // default 0.75
static final int DEFAULT_INITIAL_CAPACITY = 16; // must be power of 2
static final float DEFAULT_LOAD_FACTOR = 0.75f;
static final int TREEIFY_THRESHOLD = 8; // bucket becomes tree at >= 8 nodes
static final int UNTREEIFY_THRESHOLD = 6; // tree reverts to list when <= 6
static final int MIN_TREEIFY_CAPACITY = 64; // min table size before treeifying
}
```

Visual memory layout — 16-bucket HashMap with 5 entries:

```
table[] array (16 slots)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 0 ■ null ■
■ 1 ■ null ■
■ 2 ■ null ■
■ 3 ■ [hash=..., key="EMP-001", val=Alice, next=null] ■ <- bucket 3
■ 4 ■ null ■
■ 5 ■ [hash=..., key="EMP-002", val=Bob, next->] ■ <- COLLISION!
■ ■ ■■> [hash=..., key="EMP-005", val=Eve, next=null] ■ linked list
■ 6 ■ null ■
■ 7 ■ [hash=..., key="EMP-003", val=Carol, next=null] ■
■ 8 ■ [hash=..., key="EMP-004", val=Dan, next=null] ■
■ 9 ■ null ■
■ ... ■ null ■
■ 15 ■ null ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

Bucket 5 has a collision (linked list):
EMP-002 -> EMP-005 -> null
get("EMP-005"): go to bucket 5, walk list: check EMP-002 (no), check EMP-005 (yes!) -> Eve
After 8 collisions in same bucket -> linked list TREEIFIED to red-black tree:
O(n) worst-case list scan becomes O(log n) tree lookup
```

The put() operation — step by step:

```
// put(key, value) — simplified logic showing all steps
public V put(K key, V value) {
// STEP 1: Spread the hash to reduce clustering
int hash = (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
// Why XOR with upper 16 bits? For small arrays (16-64 buckets),
// only the lower bits of hashCode() are used for the index.
// XOR-ing upper bits INTO lower bits improves distribution.
// STEP 2: Lazy-initialize the table on first put
if (table == null || table.length == 0) resize();
// STEP 3: Calculate bucket index
int n = table.length; // e.g. 16
int i = (n - 1) & hash; // bitwise AND = fast modulo for power-of-2 sizes
// e.g. hash=1234499, n=16: index = 1234499 & 15 = 3
// STEP 4: Check the bucket
Node<K,V> p = table[i];
if (p == null) {
// STEP 4a: Bucket empty — insert directly
table[i] = newNode(hash, key, value, null);
} else {
// STEP 4b: Bucket occupied — check for existing key or collision
Node<K,V> e = null;
if (p.hash == hash && (p.key == key || (key != null && key.equals(p.key)))) {
e = p; // found existing key at head of bucket
} else if (p instanceof TreeNode) {
e = ((TreeNode<K,V>)p).putTreeVal(this, table, hash, key, value); // tree insert
} else {
// Walk the linked list
for (int binCount = 0; ; ++binCount) {
if ((e = p.next) == null) {
p.next = newNode(hash, key, value, null); // append at tail
if (binCount >= TREEIFY_THRESHOLD - 1) // 8th collision
treeifyBin(table, hash); // convert to tree!
break;
}
if (e.hash == hash && (e.key == key || (key != null && key.equals(e.key))))
break; // found existing key mid-list
p = e;
}
}
if (e != null) { // key already exists — update value
V oldValue = e.value;
e.value = value;
return oldValue;
}
}
// STEP 5: Check if resize needed
if (++size > threshold) resize(); // threshold = capacity * 0.75
return null;
}
```

The resize() / rehash operation:

```
// When size > capacity * loadFactor (default: 16 * 0.75 = 12):
// HashMap doubles the array size and REHASHES every entry.
// Before resize: capacity=16, threshold=12, 13 entries -> RESIZE triggered
// After resize: capacity=32, threshold=24
// Why must capacity be a power of 2?
// index = hash & (n-1) only works correctly as a modulo when n is a power of 2.
// n=16: n-1=15 (0b1111) -> index picks last 4 bits -> 0..15
// n=32: n-1=31 (0b11111) -> index picks last 5 bits -> 0..31
// Non-power-of-2 would cause uneven distribution.
// Rehashing cost: O(n) — all existing entries are re-indexed.
// Amortized cost of put() is O(1) because resize doubles each time.
// REAL-LIFE analogy:
// The library runs out of shelves (at 75% full). Instead of adding 1 shelf,
// they BUILD A SECOND LIBRARY with 32 shelves, move all books,
// recalculate each book's new shelf (some books change shelves because
// the ISBN calculation now uses mod 32 instead of mod 16).
// Expensive once, but happens rarely (exponentially less often as library grows).
// Pre-sizing tip: avoid rehashing if you know the expected size
Map<String, Employee> map = new HashMap<>(expectedSize * 4 / 3 + 1);
// e.g. for 1000 entries: new HashMap<>(1334) -> capacity=2048 -> no resize needed
```

4. equals() and hashCode() — The Contract

HashMap's correctness depends entirely on two Object methods: **hashCode()** and **equals()**. They form a contract that **every key class MUST honour**. Violating this contract leads to lost entries, memory leaks, and silent bugs that are extremely hard to diagnose.

The Contract (from java.lang.Object Javadoc)

1. Consistency: if x.equals(y) is true, then x.hashCode() == y.hashCode() MUST be true. 2. NOT required: if x.hashCode() == y.hashCode(), equals() may still return false (hash collision is OK). 3. Stability: multiple calls to hashCode() on the same object MUST return the same value (within one JVM run). 4. Default Object.equals() checks reference equality (==). Default Object.hashCode() is based on memory address.

Real-life analogy — the Employee ID badge:

Two employees are the 'same' if they have the same employee ID. The badge number (hashCode) determines which security gate (bucket) they use. The full ID check (equals) confirms their identity once they're at the gate. **Rule:** if two employees are the same person, they must use the same gate — but two different people can accidentally have the same gate number (collision is fine).

BROKEN class — missing equals() and hashCode():

```
// BAD: Employee uses default Object.equals() and Object.hashCode()
public class Employee {
private final String id; // e.g. "EMP-001"
private final String name; // e.g. "Alice"
public Employee(String id, String name) { this.id=id; this.name=name; }
// NO equals(), NO hashCode() overrides!
}
HashMap<Employee, String> deptMap = new HashMap<>();
Employee e1 = new Employee("EMP-001", "Alice");
deptMap.put(e1, "Engineering");
// Later in code, try to look up Alice:
Employee e2 = new Employee("EMP-001", "Alice"); // same data, different object
System.out.println(deptMap.get(e2)); // null !!! BUG!
System.out.println(e1.equals(e2)); // false (reference equality — different objects)
System.out.println(e1.hashCode() == e2.hashCode()); // false (based on memory address)
// WHAT HAPPENS INTERNALLY:
// put(e1, "Engineering"): hashCode(e1) = 0x7a3c -> bucket 4 -> stored
// get(e2): hashCode(e2) = 0x2f91 -> bucket 11 -> bucket is empty! -> null
// e1 and e2 are logically the same employee but HashMap sees them as different objects!
```

FIXED class — correct equals() and hashCode():

```
// GOOD: Employee properly overrides both methods
public final class Employee {
private final String id;
private final String name;
private final String department;
public Employee(String id, String name, String department) {
this.id=id; this.name=name; this.department=department;
}
// equals(): two employees are the same if and only if their IDs match
@Override
public boolean equals(Object obj) {
if (this == obj) return true; // same reference -> definitely equal
if (!(obj instanceof Employee)) return false; // different type -> not equal
Employee other = (Employee) obj;
return Objects.equals(this.id, other.id); // equality based on business key
}
// hashCode(): MUST be consistent with equals() — same id -> same hashCode
@Override
public int hashCode() {
return Objects.hash(id); // uses id.hashCode() with null protection
// Rule: only use fields that participate in equals()!
}
@Override
public String toString() {
return "Employee{id='" + id + "', name='" + name + "'}";
}
}
// NOW HashMap works correctly:
HashMap<Employee, String> deptMap = new HashMap<>();
Employee e1 = new Employee("EMP-001", "Alice", "Engineering");
deptMap.put(e1, "Engineering");
Employee e2 = new Employee("EMP-001", "Alice", "Engineering"); // same data
System.out.println(deptMap.get(e2)); // "Engineering" -- correct!
System.out.println(e1.equals(e2)); // true
System.out.println(e1.hashCode()==e2.hashCode()); // true -- contract satisfied
```

Implementing equals() correctly — the full checklist:

```
@Override
public boolean equals(Object obj) {
// 1. Reflexivity: x.equals(x) must be true
if (this == obj) return true;
// 2. Null safety: x.equals(null) must be false
if (obj == null) return false;
// 3. Type check: use instanceof (handles subclasses) or getClass() (exact type)
// Use instanceof if you want subclasses to be equal (Liskov-friendly)
// Use getClass() if you want exact-type equality (stricter, safer for mutable state)
if (!(obj instanceof Employee)) return false;
// OR: if (getClass() != obj.getClass()) return false;
// 4. Cast and compare ONLY the fields that define logical equality
Employee other = (Employee) obj;
return Objects.equals(this.id, other.id);
// Use Objects.equals() — handles null in both fields safely
}
// Java 16+ Records do this automatically:
public record Employee(String id, String name, String department) {}
// Record auto-generates equals() comparing ALL fields and hashCode() over ALL fields
```

hashCode() design — good vs bad implementations:

```
// BAD hashCode 1: always return same value (legal but terrible — O(n) for everything)
@Override public int hashCode() { return 42; }
// Every key hashes to bucket 42 -> entire HashMap is one giant linked list
// get() is O(n) instead of O(1) -- DEFEATS THE PURPOSE OF HASHMAP
// BAD hashCode 2: uses fields NOT in equals() (violates contract)
@Override public int hashCode() { return Objects.hash(id, name, department); }
@Override public boolean equals(Object o) { ... return Objects.equals(id, other.id); }
// Two employees with same id but different names would:
// -> equals() returns true (same id)
// -> hashCode() returns different values (different name included)
// -> CONTRACT VIOLATED: equals=true but hashCode differs
// GOOD hashCode: use EXACTLY the same fields as equals()
@Override public int hashCode() { return Objects.hash(id); } // matches equals(id)
// GOOD: multi-field equality and hash (Address object)
public class Address {
private final String street, city, zipCode;
@Override public boolean equals(Object o) {
if (!(o instanceof Address)) return false;
Address a = (Address) o;
return Objects.equals(street, a.street) &&
Objects.equals(city, a.city) &&
Objects.equals(zipCode,a.zipCode);
}
@Override public int hashCode() {
return Objects.hash(street, city, zipCode); // same 3 fields as equals
}
}
// PRO TIP: use a good prime-based multiplier for custom hashCode:
// int result = 17;
// result = 31 * result + (id != null ? id.hashCode() : 0);
// return result;
// (31 is prime, used by String.hashCode() internally — good distribution)
```

Mutable keys — the worst HashMap mistake:

```
// CRITICAL BUG: using a mutable object as HashMap key
public class MutableEmployee {
public String id; // mutable field!
// equals() and hashCode() use 'id'
}
HashMap<MutableEmployee, String> map = new HashMap<>();
MutableEmployee emp = new MutableEmployee("EMP-001");
map.put(emp, "Engineering");
System.out.println(map.get(emp)); // "Engineering" -- works
// Now mutate the key AFTER insertion:
emp.id = "EMP-999"; // CHANGES THE HASH!
System.out.println(map.get(emp)); // null !!! The key is now in the WRONG BUCKET
// hashCode() now returns a different value -> different bucket index
// The entry is still in the old bucket (bucket for "EMP-001" hash)
// but we're looking in the new bucket (for "EMP-999" hash) -> not found
// RULE: Always use IMMUTABLE objects as HashMap keys.
// String, Integer, Long, UUID, Records — all are immutable -> safe keys.
```

5. LinkedHashMap — Ordered HashMap

LinkedHashMap extends HashMap and adds a **doubly-linked list** threading through all entries in insertion order (or access order). This gives it O(1) get/put like HashMap but with predictable iteration order. It also enables a simple **LRU cache** implementation.

```
// Default: INSERTION ORDER
LinkedHashMap<String, Employee> insertionOrder = new LinkedHashMap<>();
insertionOrder.put("EMP-003", carol);
insertionOrder.put("EMP-001", alice);
insertionOrder.put("EMP-002", bob);
// Iteration order: EMP-003, EMP-001, EMP-002 (insertion order preserved)
// ACCESS ORDER: true = recently accessed moves to end (oldest first)
LinkedHashMap<String, Employee> lruCache = new LinkedHashMap<>(16, 0.75f, true);
lruCache.put("EMP-001", alice);
lruCache.put("EMP-002", bob);
lruCache.put("EMP-003", carol);
lruCache.get("EMP-001"); // accesses EMP-001 -> moves to end
// Now order: EMP-002, EMP-003, EMP-001 (EMP-001 most recently used)
// LRU Cache: evict eldest entry when capacity exceeded
int MAX_CACHE = 100;
Map<String, Employee> lruCache = new LinkedHashMap<>(MAX_CACHE, 0.75f, true) {
@Override
protected boolean removeEldestEntry(Map.Entry<String, Employee> eldest) {
return size() > MAX_CACHE; // auto-evict when size > 100
}
};
// Real-world use: session cache, DNS cache, image thumbnail cache
```

6. TreeMap — Sorted Map

TreeMap implements NavigableMap using a **red-black tree**. All keys are maintained in **sorted order** (natural ordering via Comparable, or custom order via Comparator). It provides range views, floor/ceiling/first/last operations — ideal for leaderboards, scheduling, and range queries.

```
// Natural ordering (String compareTo)
TreeMap<String, Employee> sortedByName = new TreeMap<>();
sortedByName.put("Charlie", new Employee("EMP-003","Charlie","HR"));
sortedByName.put("Alice", new Employee("EMP-001","Alice", "Eng"));
sortedByName.put("Bob", new Employee("EMP-002","Bob", "Mkt"));
// Iteration: Alice, Bob, Charlie (alphabetical)
// Custom ordering: sort employees by salary descending
TreeMap<Employee, String> sortedBySalary = new TreeMap<>(
Comparator.comparingDouble(Employee::getSalary).reversed()
);
// NavigableMap operations
sortedByName.firstKey(); // "Alice"
sortedByName.lastKey(); // "Charlie"
sortedByName.floorKey("Brad"); // "Bob" (greatest key <= "Brad")
sortedByName.ceilingKey("Brad"); // "Charlie" (smallest key >= "Brad")
sortedByName.headMap("Bob"); // {Alice=...} (keys strictly < "Bob")
sortedByName.tailMap("Bob"); // {Bob=..., Charlie=...} (keys >= "Bob")
sortedByName.subMap("Alice","Charlie"); // {Alice=..., Bob=...} (Alice inclusive, Charlie
exclusive)
sortedByName.pollFirstEntry(); // removes and returns Alice (useful for priority
processing)
// Real-world: task scheduler (TreeMap<LocalDateTime, Task> -> find next task due)
TreeMap<LocalDateTime, Task> scheduler = new TreeMap<>();
Task nextTask = scheduler.firstEntry().getValue(); // next scheduled task
```

7. ConcurrentHashMap — Thread-Safe Map

ConcurrentHashMap is the thread-safe alternative to HashMap. Java 8 redesigned it: instead of segment-level locking (Java 7), it uses **CAS (Compare-And-Swap)** for empty buckets and **synchronized on the bin head node** for non-empty buckets. Reads are fully non-blocking. No null keys or null values allowed.

```
// Thread-safe operations — safe for concurrent reads and writes
ConcurrentHashMap<String, Employee> concurrentMap = new ConcurrentHashMap<>();

// Atomic conditional operations (avoid race conditions)
// putIfAbsent: only insert if key not present (atomic check-then-act)
concurrentMap.putIfAbsent("EMP-001", new Employee("EMP-001", "Alice", "Eng"));

// computeIfAbsent: create and insert if absent (lazy initialization, atomic)
Employee emp = concurrentMap.computeIfAbsent("EMP-001",
id -> employeeRepo.findById(id).orElseThrow());

// compute: atomically update a value based on current value
concurrentMap.compute("EMP-001", (key, existing) ->
existing == null ? new Employee(key, "Unknown", "?") :
existing.withDepartment("NewDept"));

// merge: combine old and new values atomically
concurrentMap.merge("EMP-001", newEmployee,
(oldEmp, newEmp) -> oldEmp.updated(newEmp));

// Word frequency counter — classic ConcurrentHashMap use case
ConcurrentHashMap<String, Integer> wordCount = new ConcurrentHashMap<>();
words.parallelStream().forEach(word ->
wordCount.merge(word, 1, Integer::sum)); // thread-safe increment

// WHY NOT Collections.synchronizedMap(new HashMap<>())?
// synchronizedMap wraps ALL operations in synchronized(map) — serialises everything
// ConcurrentHashMap allows concurrent reads (no lock) + fine-grained write locks
// synchronizedMap: one thread at a time; ConcurrentHashMap: N threads simultaneously
```

8. Map API — Key Methods with Examples

```
Map<String, Employee> map = new HashMap<>();
// ■■ Basic operations ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
map.put("EMP-001", alice); // insert or replace
map.get("EMP-001"); // returns alice, or null if absent
map.getOrDefault("EMP-999", defaultEmp); // returns defaultEmp if key absent
map.containsKey("EMP-001"); // true
map.containsValue(alice); // O(n) scan — avoid in hot paths
map.remove("EMP-001"); // removes entry, returns old value
map.size(); // number of entries
map.isEmpty(); // true if size == 0
map.clear(); // removes all entries
// ■■ Conditional put / update ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
map.putIfAbsent("EMP-001", alice); // only insert if key not present
map.replace("EMP-001", updatedAlice); // only update if key present
map.replace("EMP-001", alice, updatedAlice); // only update if CURRENT value matches
// ■■ Compute variants (atomic in ConcurrentHashMap) ■■■■■■■■■■■■■■■■■■■■■■■■
map.computeIfAbsent("EMP-002", k -> loadFromDB(k)); // insert if absent
map.computeIfPresent("EMP-001", (k,v) -> v.promote()); // update if present
map.compute("EMP-001", (k,v) -> v == null ? newEmp(k) : v.withDept("Eng")); // always
// ■■ merge: combine old and new value ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
map.merge("EMP-001", alice, (old, nw) -> old.merge(nw)); // custom merge logic
// ■■ Iteration ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Option 1: entrySet() — most efficient (access key AND value)
for (Map.Entry<String, Employee> entry : map.entrySet()) {
System.out.println(entry.getKey() + " -> " + entry.getValue());
}
// Option 2: forEach lambda
map.forEach((id, emp) -> System.out.println(id + ": " + emp.getName()));
// Option 3: keySet() then get() — AVOID (two lookups per entry)
for (String key : map.keySet()) { map.get(key); } // inefficient
// ■■ Views ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Set<String> keys = map.keySet(); // live view — changes reflect in map
Collection<Employee> vals = map.values(); // live view
Set<Map.Entry<String,Employee>> entries = map.entrySet(); // live view
```

9. Interview Q&A: — Standard Questions

Q1. How does HashMap store key-value pairs internally?

HashMap uses an array of Node objects called table (default size 16). Each Node stores: the key, value, the hash of the key, and a next pointer for chaining. When you call put(key, value): (1) compute hash = hashCode(key) ^ (hashCode(key) >>> 16) to spread bits; (2) find bucket index = hash & (table.length - 1); (3) if bucket is empty, insert directly; (4) if occupied, walk the linked list checking equals(); if key found, update value; if not found, append new node; (5) if any bucket's list reaches 8 nodes, convert it to a red-black tree. If total entries exceed 75% of table.length, resize (double the array).

Q2. What is the contract between equals() and hashCode()?

The contract has two rules: (1) if a.equals(b) is true, then a.hashCode() MUST equal b.hashCode() — this is mandatory for HashMap correctness. (2) if a.hashCode() == b.hashCode(), equals() may still return false — hash collisions are allowed. Additionally, hashCode() must be consistent: repeated calls return the same value within a JVM run. Violating rule 1 means two logically equal keys hash to different buckets — get() will return null even though the key was put() into the map.

Q3. What happens if you use a mutable object as a HashMap key?

Using a mutable key is a critical bug. When you put(key, value), the key's hashCode() determines the bucket. If you later mutate the key's fields that hashCode() uses, the hash changes — so the entry is now in the wrong bucket. A subsequent get(key) computes the new hash, looks in the new bucket, finds nothing, and returns null. The entry is orphaned — unreachable but not garbage-collected. Always use immutable objects as keys: String, Integer, Long, UUID, Enum, Records.

Q4. What is hash spreading and why does HashMap do it?

HashMap computes: int h = key.hashCode(); hash = h ^ (h >>> 16). This XORs the upper 16 bits of the hash into the lower 16 bits. Without this, for small tables (16 or 32 buckets), the bucket index only uses the last 4 or 5 bits of hashCode(). Many hash functions produce poor distribution in the lower bits — two keys with very different hashCodes might end up in the same bucket. Spreading prevents this by mixing upper and lower bits, improving distribution even for small tables.

Q5. What is load factor and why is the default 0.75?

Load factor is the ratio of entries to buckets that triggers a resize. At the default 0.75, a 16-bucket HashMap resizes when 12 entries are added. 0.75 balances time (fewer collisions = faster lookups) and space (smaller arrays waste less memory). Lower load factor (e.g., 0.5) means more memory but fewer collisions. Higher load factor (e.g., 0.9) saves memory but increases collision rate and degrades to O(n) lookups. 0.75 is empirically shown to be near-optimal for typical hash functions.

Q6. Why must HashMap capacity always be a power of 2?

The bucket index is calculated as: index = hash & (capacity - 1). This bitwise AND works as a fast modulo operation ONLY when capacity is a power of 2. For capacity=16: capacity-1=15 (0b1111), so & 15 extracts the last 4 bits, giving 0..15 uniformly. For non-power-of-2 (e.g., 17: capacity-1=16=0b10000), only bit 4 would be used, collapsing all entries to bucket 0 or 16 — terrible distribution. Powers of 2 ensure uniform distribution and fast calculation.

Q7. What happens when two keys have the same hashCode?

They end up in the same bucket — this is a hash collision. HashMap handles collisions with chaining: the bucket becomes a linked list of Node entries. On get(), HashMap walks the list checking both hash equality AND key.equals() to find the right entry. Before Java 8, large buckets could degrade to O(n). Java 8+ treeifies any bucket with 8+ nodes into a red-black tree, giving O(log n) worst case. When nodes drop to 6 (via removal), the tree converts back to a list.

Q8. What is the difference between HashMap and Hashtable?

Hashtable is legacy (Java 1.0): all methods are synchronized, allowing null neither for keys nor values. HashMap was introduced in Java 2: not synchronized, allows one null key and multiple null values, faster in single-threaded use. Hashtable's synchronization is coarse-grained (locks the entire table per operation) — poor for multi-threaded use. Modern code should use HashMap (single-threaded) or ConcurrentHashMap (multi-threaded). Hashtable is considered obsolete.

Q9. How does TreeMap maintain sorted order?

TreeMap uses a red-black tree — a self-balancing binary search tree. Keys are ordered by their natural ordering (Comparable.compareTo()) or by a custom Comparator injected at construction. On put(), the key is compared and placed in the correct position to maintain the BST invariant. On get(), a binary search finds the key in O(log n). Because keys are sorted, TreeMap supports range operations: subMap(), headMap(), tailMap(), floorKey(), ceilingKey(), firstKey(), lastKey(). TreeMap does not use hashCode() at all.

Q10. What is the difference between HashMap and LinkedHashMap?

LinkedHashMap extends HashMap and adds a doubly-linked list connecting all entries in either insertion order (default) or access order. This adds a small constant overhead per entry (two extra pointers) but provides predictable iteration order. HashMap iterates in an undefined order (depends on hash values and bucket distribution). LinkedHashMap is used when order matters: ordered logs, LRU caches (with accessOrder=true and overriding removeEldestEntry), deterministic test data.

Q11. How does ConcurrentHashMap achieve thread-safety without full locking?

In Java 8+, ConcurrentHashMap uses CAS (Compare-And-Swap) for inserting into empty buckets (no lock needed — just an atomic memory operation). For non-empty buckets (collision case), it synchronizes only on the head node of that bucket — so writes to different buckets proceed concurrently. Reads are fully non-blocking (volatile reads). This gives much better throughput than synchronizedMap (which locks the entire map per operation). The result: O(1) concurrent reads and fine-grained locked writes.

Q12. Why does ConcurrentHashMap not allow null keys or values?

In a single-threaded HashMap, map.get(key) returning null is unambiguous: the key is absent. In a concurrent context, it's ambiguous: get() returning null could mean (1) the key was absent when checked, or (2) a concurrent thread just inserted null. To safely distinguish them you'd need containsKey(key), but the state might change between the two calls. Disallowing null eliminates the ambiguity. Doug Lea (author of ConcurrentHashMap) stated this was a deliberate design decision: 'null in a concurrent map is inherently ambiguous'.

Q13. What is the difference between putIfAbsent() and computeIfAbsent()?

putIfAbsent(key, value) always constructs the value object even if the key is already present (wasted object creation). computeIfAbsent(key, mappingFunction) only invokes the mapping function if the key is absent — the value is computed lazily. For expensive values (DB lookups, new object construction), computeIfAbsent is preferred: it avoids unnecessary work. Both are atomic in ConcurrentHashMap. Example: cache.computeIfAbsent(userId, id -> loadFromDB(id)) — only queries DB on cache miss.

Q14. What is an EnumMap and when should you use it?

EnumMap is a Map implementation backed by an array indexed by Enum ordinal values. Since enum constants are numbered 0..n, the array lookup is O(1) with no hashing needed. EnumMap is faster, more memory-efficient, and maintains enum declaration order. Use EnumMap when all keys are enum values: status -> count, day -> schedule, role -> permissions. Never use HashMap when EnumMap is available — EnumMap is always faster for enum keys.

Q15. What is WeakHashMap and when is it useful?

WeakHashMap stores keys as WeakReferences — keys can be garbage collected if no other strong reference to the key exists. When a key is GC'd, the corresponding entry is removed from the map. Use case: caches where the key is a heavyweight object (e.g., a ClassLoader, a Widget) and the cache should not prevent garbage collection. If you hold a strong reference to the key elsewhere, the entry stays. If you release all strong references, the entry disappears. Contrast with regular HashMap which always holds a strong reference to the key.

Q16. How would you implement an LRU cache using LinkedHashMap?

Create a LinkedHashMap with accessOrder=true and override removeEldestEntry() to return true when size > maxCapacity. Access-order mode moves the most recently accessed entry to the end, so the oldest (least recently used) entry is always at the front. removeEldestEntry() is called after each put() and can evict the first (eldest) entry. The result is a bounded LRU cache in about 10 lines of code: new LinkedHashMap<>(maxCapacity, 0.75f, true) { protected boolean removeEldestEntry(Entry eldest) { return size() > maxCapacity; } }.

Q17. What is the time complexity of HashMap operations?

Average case: put() O(1), get() O(1), remove() O(1), containsKey() O(1). Worst case (all keys hash to same bucket): O(n) for linked list, O(log n) for treeified bucket (Java 8+, bucket size >= 8). Iteration over all entries: O(n + capacity) — iterates all buckets including empty ones, so a large initial capacity with few entries wastes iteration time. Space: O(n) for entries + O(capacity) for the bucket array.

Q18. How does the merge() method work in Map?

merge(key, value, remappingFunction) works as follows: if the key is absent or mapped to null, it inserts value. If the key is present, it applies remappingFunction(oldValue, value) and: if the result is non-null, updates the mapping; if null, removes the key. Use case: word frequency counting — map.merge(word, 1, Integer::sum) inserts 1 on first occurrence, then adds 1 to the existing count on each subsequent occurrence. Cleaner than the putIfAbsent/compute alternatives for simple accumulation.

Q19. What does entrySet() return and why is it the preferred iteration approach?

entrySet() returns a Set> — a live view of the map's entries. Iterating over entrySet() gives both key and value in each iteration step without a second lookup. Contrast with keySet() iteration where you'd call map.get(key) for each key — two map operations per entry. entrySet() is always preferred for iteration when you need both key and value. map.forEach((k,v) -> ...) is the most concise form and is equivalent to entrySet() iteration.

Q20. What is the difference between remove(key) and remove(key, value)?

remove(key) removes the entry for the key unconditionally, returning the old value (or null). remove(key, value) is a conditional remove: it only removes the entry if the key is currently mapped to exactly that value (checked via equals()). Returns true if removed, false if not. Useful for atomic conditional removal in ConcurrentHashMap: only one thread wins the race to remove a specific key-value pair. Example: cache.remove(key, expiredValue) — removes only if the cached value hasn't been updated.

Q21. How should you handle a HashMap in a multi-threaded environment?

Options: (1) ConcurrentHashMap — best for high concurrency; fine-grained locking; allows concurrent reads; no full-table lock. (2) Collections.synchronizedMap(new HashMap<>()) — coarse synchronization; all operations serialised; iteration must also be in synchronized block. (3) Use a concurrent data structure (Guava Cache, Caffeine) for caching use cases with eviction, expiry, stats. (4) Confine the map to a single thread (thread-local or immutable after construction). Never use a plain HashMap from multiple threads without synchronization — data races can corrupt the internal linked list, causing infinite loops during rehashing.

Q22. Why can iterating a HashMap from multiple threads cause an infinite loop?

In Java before 7, the resize() method used a naive reversal of the linked list in each bucket. Two threads both resizing simultaneously could create a cycle in a bucket's linked list — a circular linked list. The next call to get() on that bucket would loop forever. This is a classic Java HashMap race condition. Java 8's resize was redesigned to avoid reversals. However, even with Java 8+ HashMap, concurrent modification causes undefined behaviour — use ConcurrentHashMap instead.

Q23. What is the difference between a HashMap key being absent and being mapped to null?

A HashMap allows one null key and null values. map.get(key) returning null is ambiguous: it could mean the key is absent, or the key maps to null. Use map.containsKey(key) to distinguish: if containsKey returns true but get returns null, the key explicitly maps to null. If containsKey returns false, the key is absent. Modern code avoids null values: use Optional values, or Map.getOrDefault(key, defaultValue). ConcurrentHashMap eliminates this ambiguity by prohibiting null keys and values.

Q24. What is the Map.of() factory method and how does it differ from new HashMap<>()?

Map.of() (Java 9+) creates an immutable map with up to 10 key-value pairs. Map.ofEntries() handles more. The resulting map: throws UnsupportedOperationException on any mutating operation (put, remove, clear), does not allow null keys or values, has unspecified iteration order, and is not serializable in the traditional sense. Use Map.of() for configuration maps, test fixtures, and lookup tables that should never change. new HashMap<>() creates a mutable map suitable for building and modifying. Map.copyOf() creates an immutable copy of an existing map.

Q25. How does Objects.hash() work and is it suitable for hashCode()?

Objects.hash(a, b, c) calls Arrays.hashCode(new Object[]{a, b, c}) which computes: result = 1; for each element: result = 31 * result + (element == null ? 0 : element.hashCode()). It handles null elements safely (returns 0 for null). It's convenient and correct for most cases. The slight drawback: it creates a varargs Object[] array on each call — for performance-critical hot paths, manually compute the hash with the 31* formula. For most application code, Objects.hash() is the recommended approach.

Q26. Describe how HashMap handles the null key.

HashMap allows exactly one null key. Null keys are always placed in bucket 0 (index 0) because: if key == null, hashCode() cannot be called (NullPointerException), so HashMap hard-codes null to hash=0 and bucket 0. put(null, value) stores the entry at table[0]. get(null) looks in table[0] and compares with null using == (since null == null is true). ConcurrentHashMap and TreeMap do not support null keys: ConcurrentHashMap would make containsKey ambiguous; TreeMap would throw NullPointerException when trying to call compareTo(null).

10. Interview Q&A: — Advanced Questions

Advanced Section

Senior/Lead/Architect-level questions on JVM internals, concurrent maps, design trade-offs, and HashMap edge cases.

[ADV] Q27. Explain the treeification threshold and why it is 8.

Java 8 treeifies a bucket when it reaches 8 nodes (TREEIFY_THRESHOLD=8). This threshold balances overhead: a red-black tree node is about twice the size of a linked list node (TreeNode has 6 extra fields). For small buckets, the list's O(n) cost is acceptable and memory-efficient. Poisson distribution analysis shows that under uniform hashing with load factor 0.75, the probability of a bucket reaching 8 entries is approximately 6 in 10 million — extremely rare. When it does occur (hash collision attack or poor hashCode), the tree provides O(log n) rather than O(n) safety. UNTREEIFY_THRESHOLD=6 avoids thrashing (repeatedly tree-ify and un-treeify).

[ADV] Q28. What is a hash flood / hash collision attack and how does Java defend against it?

A hash flood attack deliberately crafts input keys that all hash to the same bucket, turning HashMap lookups into O(n) scans and causing DoS. In Java before 7, String.hashCode() was deterministic — attackers could precompute collision strings. Java 7 introduced hash randomization (ALTERNATIVE_HASHING) for String keys in Maps when the table is large. Java 8's treeification is the main defence: even if an attacker floods one bucket, the tree cap at O(log n) prevents the DoS. Additionally, using secure hash functions (e.g., SipHash in Python, Rust) fully mitigates hash flooding.

[ADV] Q29. How does ConcurrentHashMap's computeIfAbsent guarantee atomicity?

For an empty bucket, ConcurrentHashMap uses CAS (compareAndSet on the bin's volatile reference): only one thread successfully sets the new node; others retry the CAS loop. For non-empty buckets, it acquires synchronized(binHead) before calling the mapping function. This means the mapping function runs while holding the bin lock — it should be fast and non-blocking. If the mapping function itself calls back into the same ConcurrentHashMap (recursive computeIfAbsent on the same key), it will deadlock because the lock is not reentrant for this operation. Java 9+ throws IllegalStateException in this case.

[ADV] Q30. What is the performance impact of a bad hashCode() implementation?

If all keys return the same hashCode() (e.g., return 42), every entry lands in bucket 0. The map degrades to a single linked list (O(n) get/put) or red-black tree (O(log n)) after 8 entries. For 1 million entries: expected O(1) get becomes O(log 1000000) ≈ 20 comparisons — a 20x slowdown. For read-heavy workloads with 10,000 gets/second, this adds significant latency. A good hashCode() distributes keys uniformly across all buckets, keeping average bucket size = total_entries / capacity ≈ 0.75 at steady state.

[ADV] Q31. How does equals() interact with instanceof vs getClass() — which should you use?

getClass() check: only returns true for exact same class — a subclass cannot equal a superclass. This is required when the subclass adds fields that affect equality (Liskov substitution: a Point(1,1) equals a ColorPoint(1,1,RED)? With getClass(), no). instanceof check: allows subclasses to be equal — better for interfaces and abstract classes. The general rule: if the class is final or its subclasses cannot add equality-relevant fields, instanceof is fine. For mutable domain objects in JPA entities, use getClass() to avoid Hibernate proxy equality issues. Hibernate proxies are subclasses of your entity — instanceof returns true for the proxy but the proxy may not equal the real entity.

[ADV] Q32. How do JPA/Hibernate entities interact with HashMap's equals/hashCode contract?

Hibernate lazy-loads entities as proxy subclasses. If your entity uses the JPA-generated @Id as the hashCode, there's a bootstrap problem: a new (transient) entity has id=null, giving hashCode=0. After persist(), id=1, giving a different hashCode. If the entity was stored in a Set before persist(), it's now in the wrong bucket — lost forever. Solutions: (1) Use a business key (@NaturalId) for equals/hashCode — stable across lifecycle. (2) Use UUID assigned in the constructor (before persist). (3) Use a constant hashCode (return 1) — correct but O(n). Hibernate's own recommendation: business-key equality or UUID.

[ADV] Q33. What is the difference between Map.Entry.setValue() and map.put() during iteration?

During iteration via entrySet(), you MAY call entry.setValue(newValue) to update the value in-place — this modifies the map without changing the structural layout (no modCount increment for HashMap, safe for iteration). Calling map.put() or map.remove() during iteration modifies modCount, causing ConcurrentModificationException in fail-fast iterators. For thread-safe in-place value updates during iteration, use entrySet().iterator() + entry.setValue(). For adding/removing keys, use a separate pass or CopyOnWriteArrayList equivalent.

[ADV] Q34. How does HashMap handle Integer keys vs String keys differently?

Integer.hashCode() returns the int value itself (hashCode of Integer(42) = 42). For small integers as keys, the hash distribution depends entirely on how many integers are used and their range. Consecutive integers 0..15 hash to buckets 0..15 perfectly. Large integers (e.g., user IDs like 1000001..1000016) would all have the same lower 4 bits after & 15 — poor distribution. String.hashCode() uses the polynomial 31*h + c formula over all characters — better distribution for arbitrary strings. The hash spreading step (h ^ h>>>16) helps both, but String keys generally distribute better than sequential integers.

[ADV] Q35. How would you design a thread-safe, expiry-based cache using ConcurrentHashMap?

Store CacheEntry objects as values: record CacheEntry(V value, long expiryMs). computeIfAbsent for cache population. For expiry: in get(), check System.currentTimeMillis() > entry.expiryMs — if expired, remove and return null (or refresh). Use scheduled EvictionTask (ScheduledExecutorService) to periodically scan and remove expired entries in a background thread. Use compute() to atomically check-and-refresh. For production: use Caffeine (Google Guava's successor) which does this with precise timing, weak keys, and metrics. Caffeine's ConcurrentHashMap-backed implementation is battle-tested.

[ADV] Q36. Explain how resizing works when two threads simultaneously trigger it.

Before Java 8, two threads resizing simultaneously could create circular linked lists in buckets — an infinite loop on the next get(). Java 8 redesigned resize() to use a split approach: entries whose (hash & oldCapacity) == 0 stay in the same bucket index in the new table (lo chain); others move to index + oldCapacity (hi chain). The list is never reversed. However, this is still not thread-safe — concurrent resize in a plain HashMap is undefined behaviour. The fix: use ConcurrentHashMap, which has a coordinated resize protocol using a transferIndex field and sizeCtl flag to safely distribute resize work across threads.

[ADV] Q37. What is a NavigableMap and what operations does it add over SortedMap?

NavigableMap extends SortedMap and adds: lowerKey(k)/floorKey(k) (greatest key < k / <= k), ceilingKey(k)/higherKey(k) (smallest key >= k / > k), pollFirstEntry()/pollLastEntry() (retrieve and remove first/last), descendingMap() (reverse-order view), descendingKeySet(), subMap(fromKey, fromInclusive, toKey, toInclusive) with inclusion/exclusion control. TreeMap implements NavigableMap. Use case: a leaderboard (find the 10 players just below a given score), a scheduler (find the next task due after a given time), an IP routing table (find the longest prefix match).

[ADV] Q38. How does the Map.compute() method differ from a get-then-put pattern?

get-then-put: V old = map.get(key); V nw = transform(old); map.put(key, nw) — three operations, NOT atomic. Between get and put, another thread might change the value, causing a lost update. compute(key, fn) atomically reads the current value and writes the result of fn(key, currentValue) in a single locked operation (in ConcurrentHashMap). This eliminates the check-then-act race condition. Use compute() for all atomic read-modify-write operations on concurrent maps. Also handles null: fn receives null if key is absent; if fn returns null, the key is removed.

[ADV] Q39. What are the implications of overriding only equals() without hashCode()?

If you override equals() but not hashCode(), two logically equal objects will have different hashCodes (using Object's identity-based default). When used as HashMap keys: put(obj1, value) hashes to bucket A; get(obj2) (where obj2.equals(obj1)) hashes to a different bucket B — get returns null. The entry exists in the map but is unreachable via get(). Additionally, a Set would allow duplicates (because HashSet uses hashCode to find the bucket first). This is one of the most common Java bugs — IDEs and static analysis tools (FindBugs, SpotBugs, SonarQube) have dedicated rules for this violation.

[ADV] Q40. How does Java's String interning interact with HashMap lookups?

String.intern() returns the canonical instance from the String pool. Two interned strings with the same content have the same reference (==). This does NOT change HashMap behaviour: HashMap uses hashCode() + equals() regardless of reference. Using interned strings as keys does not make HashMap faster — it was already using String.hashCode() (which is the same for equal strings whether interned or not). Interning CAN benefit memory (one shared String object) but NOT HashMap lookup speed. Where it DOES help: using == instead of .equals() in custom equals() implementations — but this is an anti-pattern; always use .equals() for String comparison.

[ADV] Q41. How would you count word frequencies using Map in parallel?

Use ConcurrentHashMap.merge(): words.parallelStream().forEach(word -> freq.merge(word, 1, Integer::sum)). merge() is atomic in ConcurrentHashMap. Alternatively, Collectors.groupingByConcurrent(word -> word, Collectors.counting()) in a parallel stream. For single-threaded: Collectors.groupingBy(word -> word, Collectors.counting()) into a HashMap. Performance note: parallelStream with ConcurrentHashMap.merge has high contention for frequent words — use Collectors.toConcurrentMap with partitioned approaches (split work by first character) for extreme throughput.

[ADV] Q42. What is the space overhead of HashMap vs array for storing 1 million int-keyed entries?

Array (int[] values, index=key): 4MB for 1M ints. HashMap: each Integer key = 16 bytes (object header) + 4 bytes int = 20 bytes; each Node = 16 (header) + 4 (hash) + 8 (key ref) + 8 (value ref) + 8 (next ref) = 44 bytes; plus array capacity (at 75% load, array = 1.33M buckets = 1.33M * 8 bytes refs = ~10MB). Total ~64MB+ — roughly 16x overhead vs array. Lesson: for dense integer-keyed maps, arrays beat HashMap. For sparse or non-integer keys, HashMap is the right choice. EnumMap uses an array internally, giving near-array performance.

[ADV] Q43. How would you create an immutable map and what are its performance characteristics?

Java 9+ Map.of(k1,v1,...) and Map.copyOf(map) create immutable maps. Internally, Map.of() uses a compact array representation with open addressing (no linked list overhead) — smaller memory than HashMap and faster iteration. For 1-10 entries, Map.of() outperforms HashMap in both memory and lookup due to array scan being cache-friendly. For larger maps, HashMap's O(1) hash lookup beats array scan. Collections.unmodifiableMap(hashMap) wraps a mutable map — still backed by HashMap memory layout, just throws on mutations. For true immutability and best performance, prefer Map.of() / Guava ImmutableMap.

[ADV] Q44. Explain how TreeMap handles Comparable vs Comparator ordering.

TreeMap uses keys' natural ordering (Comparable.compareTo()) by default. If a Comparator is provided to the constructor, it is used instead. Important: TreeMap's ordering must be consistent with equals() — that is, compareTo()/compare() should return 0 if and only if equals() returns true. If they disagree (e.g., a Comparator that sorts case-insensitively but equals() is case-sensitive), contains() and get() may behave unexpectedly: the tree finds the 'equal' key by comparison but equals() returns false, causing confusing results. The Javadoc states this strongly: 'The ordering must be consistent with equals.'

[ADV] Q45. How does HashMap's iteration order change between JVM runs?

HashMap's iteration order depends on: hashCodes of keys (determined by key.hashCode() implementation), the hash spreading, and the current table capacity. For String keys: String.hashCode() is deterministic within a JVM run (always returns the same value for the same string). Iteration order is consistent within a JVM run but may differ between JVM versions (if the hash algorithm changed). For Integer keys: Integer.hashCode() = the integer value — deterministic and consistent. The order is NOT guaranteed to be consistent across JVM restarts, and the JDK documentation explicitly warns against relying on HashMap iteration order. Use LinkedHashMap or TreeMap when order matters.

[ADV] Q46. What happens if hashCode() returns Long.MAX_VALUE for all keys?

int hash = (h = key.hashCode()) ^ (h >>> 16) where h = Long.MAX_VALUE cast to int. hashCode() returns int in Java (not long), so Long.MAX_VALUE as a hashCode is impossible — it wraps to Integer.MAX_VALUE = 2147483647. Hash spreading: 2147483647 ^ (2147483647 >>> 16) = 2147483647 ^ 32767 = 2147516416. Bucket index for 16 buckets: 2147516416 & 15 = 0. All keys land in bucket 0 — O(n) or O(log n) for all operations. This illustrates why poor hashCode implementations (all same value or very similar values) destroy HashMap performance.

[ADV] Q47. How does computeIfAbsent enable safe lazy initialisation of nested Maps?

Pattern: Map>> org = new HashMap<>(); // nested map. Without computeIfAbsent: if (!org.containsKey(dept)) org.put(dept, new HashMap<>()); org.get(dept).computeIfAbsent(role, k -> new ArrayList<>()).add(employee); — three separate operations. With computeIfAbsent: org.computeIfAbsent(dept, k -> new HashMap<>()).computeIfAbsent(role, k -> new ArrayList<>()).add(employee); — creates inner maps lazily and atomically. In ConcurrentHashMap, this is fully thread-safe: only one thread creates the inner map even under concurrency.

[ADV] Q48. What is the difference between keySet(), values(), and entrySet() as live views?

All three return live views backed by the underlying map. Changes to the map are reflected in the view; changes to the view (removes only — adds are not supported) are reflected in the map. keySet().remove(k) removes the entire entry. values().remove(v) removes one entry mapping to v (arbitrary if multiple). entrySet().remove(entry) removes the specific entry. You can iterate and remove via the view's iterator.remove(). You cannot add to keySet() or values() — throws UnsupportedOperationException. For ConcurrentHashMap, the views support concurrent modification without ConcurrentModificationException (weakly consistent).

[ADV] Q49. How would you design equals() for a JPA entity to work correctly with Hibernate proxies?

Hibernate creates CGLIB/ByteBuddy proxy subclasses for lazy-loaded entities. Use instanceof (not getClass()) to allow proxy equals real entity: if (!(other instanceof MyEntity)) return false. Then: MyEntity o = (MyEntity) other; — this works even if other is a proxy subclass. Use Hibernate.getClass(this) and Hibernate.getClass(other) from hibernate-core to get the real class (unwrapping proxies). For the equality field, use a @NaturalId business key (email, username, product code) rather than the database-generated @Id. If using @Id, ensure it's assigned before equals() is ever called (e.g., UUID in constructor).

[ADV] Q50. What is the Guava ImmutableMap and how does it differ from Map.of()?

Both create immutable maps. Guava ImmutableMap: supports any number of entries (Map.of() is limited to 10 pairs), uses a builder API (ImmutableMap.builder().put(k,v)...build()), maintains iteration in insertion order, has a compact internal representation (two arrays: keys and values with open-addressing hash table), is serializable, and has null-hostile behaviour with clear error messages. Map.of() (JDK 9+): limited to 10 key-value pairs inline (Map.ofEntries() for more), randomizes iteration order for security (JEP 269), is lighter-weight with no external dependency. For production: Guava ImmutableMap when insertion order matters or >10 entries; Map.of() for small ad-hoc config maps.

[ADV] Q51. How does the JVM optimise HashMap access for Integer keys through autoboxing?

Autoboxing converts int -> Integer on every Map.put()/get() call. Integer.valueOf() caches -128..127 (Flyweight pattern) so cached integers are reused without allocation. For IDs outside this range (typical DB IDs like 1001..1000000), each get(1001) autoboxes a new Integer(1001) object — potential GC pressure in hot paths. Mitigations: (1) Use primitive-specialised maps from Eclipse Collections, Koloboke, or Agrona (IntObjectHashMap) that avoid boxing entirely. (2) Cache commonly used Integer keys. (3) Use long/String keys where boxing overhead is acceptable. JIT can sometimes elim eliminate boxing via escape analysis when the boxed object doesn't escape the method.

[ADV] Q52. Design a HashMap-backed cache that is both size-bounded and time-bounded.

Combine LinkedHashMap (size bounding via removeEldestEntry) with CacheEntry(value, expiryNanos). On get(): check expiry — if expired, remove and return null. On put(): store CacheEntry with expiryNanos = System.nanoTime() + ttlNanos; LinkedHashMap's removeEldestEntry bounds size. A background ScheduledExecutorService scans and removes expired entries every N seconds to prevent memory accumulation from never-accessed expired entries. For thread-safety, wrap in Collections.synchronizedMap() or reimplement with ConcurrentHashMap + ConcurrentLinkedQueue for deferred removal. Production choice: Caffeine cache (@Caffeine(maximumSize=1000, expireAfterWrite=60s)) implements exactly this with optimal performance.

11. Coding Exercises

Exercise 1: Fix the broken Employee HashMap

- Given: class Employee { String id; String name; } stored in a HashMap.
- Problem: get() always returns null even when the same logical employee is looked up.
- Task: implement correct equals() and hashCode() using 'id' as the business key.
- Verify: e1.equals(e2) == true, hashCode equality, HashMap.get() returns correct department.
- Bonus: make Employee a Java Record and observe that equals/hashCode are generated automatically.

Exercise 2: LRU Session Cache

- Implement SessionCache backed by LinkedHashMap with a maximum of 500 sessions.
- Evict the least recently USED session (not inserted) when the limit is reached.
- Thread-safe: wrap in Collections.synchronizedMap or implement with ReentrantLock.
- Test: insert 501 sessions; verify the least recently accessed session is evicted.

Exercise 3: Word Frequency Counter

- Read a large text file and count word frequencies using ConcurrentHashMap.
- Use merge(word, 1, Integer::sum) for atomic frequency counting.

- Find the top 10 most frequent words using entrySet() + stream().sorted().
- Bonus: parallelise the word counting across chunks using parallelStream() + groupingByConcurrent().

12. Debug Scenarios

Debug 1: Lost map entry after key mutation

```
// BUG: mutable key mutated after insertion
List<String> key = new ArrayList<>(List.of("Engineering"));
Map<List<String>, Employee> map = new HashMap<>();
map.put(key, alice);
key.add("Backend"); // mutates the key!
System.out.println(map.get(key)); // null! Entry lost.
// WHY: List.hashCode() depends on its contents.
// Original hash: based on ["Engineering"] -> bucket 5
// After mutation: hash based on ["Engineering","Backend"] -> different bucket
// get() looks in new bucket -> not found
// FIX: use immutable keys
Map<List<String>, Employee> map = new HashMap<>();
map.put(List.of("Engineering"), alice); // List.of() is immutable
// OR use a String key instead:
map.put("Engineering", alice);
```

Debug 2: equals() without hashCode() — duplicate Set entries

```
// BUG: overrides equals() but NOT hashCode()
public class Product {
String sku;
@Override public boolean equals(Object o) {
return o instanceof Product && ((Product)o).sku.equals(this.sku);
}
// NO hashCode() override!
}
Set<Product> catalog = new HashSet<>();
catalog.add(new Product("SKU-001"));
catalog.add(new Product("SKU-001")); // should be duplicate!
System.out.println(catalog.size()); // 2 !!! BUG — duplicates allowed
// Each Product gets Object's identity-based hashCode -> different buckets
// HashSet never finds the "existing" entry -> both inserted
// FIX:
@Override public int hashCode() { return Objects.hash(sku); }
```

Debug 3: ConcurrentModificationException during iteration

```
// BUG: removing entries from map while iterating with for-each
Map<String, Employee> map = new HashMap<>(getEmployees());
for (Map.Entry<String, Employee> entry : map.entrySet()) {
if (entry.getValue().isInactive()) {
map.remove(entry.getKey()); // BUG: ConcurrentModificationException!
}
}
// FIX 1: use iterator.remove() -- safe structural modification during iteration
Iterator<Map.Entry<String, Employee>> it = map.entrySet().iterator();
while (it.hasNext()) {
if (it.next().getValue().isInactive()) it.remove(); // safe!
}
// FIX 2: Java 8 removeIf on entrySet
map.entrySet().removeIf(e -> e.getValue().isInactive());
// FIX 3: collect keys to remove, then removeAll
Set<String> toRemove = map.entrySet().stream()
.filter(e -> e.getValue().isInactive())
.map(Map.Entry::getKey).collect(Collectors.toSet());
toRemove.forEach(map::remove);
```

13. Multiple Choice Questions

1. What is the default initial capacity and load factor of a HashMap?

- A) 8 and 0.75
- B) 16 and 0.75 ✓
- C) 16 and 1.0
- D) 32 and 0.75

2. If equals() returns true for two objects, what MUST be true about their hashCodes?

- A) They must be different
- B) They must be equal ✓
- C) They may be equal or different
- D) They must both be zero

3. HashMap treeifies a bucket when its size reaches:

- A) 4
- B) 6
- C) 8 ✓
- D) 16

4. Which Map implementation maintains keys in sorted order?

- A) HashMap
- B) LinkedHashMap
- C) TreeMap ✓
- D) ConcurrentHashMap

5. Why should mutable objects NOT be used as HashMap keys?

- A) HashMap only accepts String keys
- B) Mutating the key changes its hashCode, causing the entry to be unreachable ✓
- C) Mutable keys cause ConcurrentModificationException
- D) HashMap copies keys on insertion

6. Which method atomically inserts a value into a ConcurrentHashMap only if the key is absent AND computes the value lazily?

- A) putIfAbsent()

- B) merge()
- C) computeIfAbsent() ✓
- D) replace()

14. Quick Reference Cheat Sheet

Concept	Key Rule / Structure	Example / Use Case
HashMap structure	Node[] table array + linked list (or tree) per bucket. Default 16 buckets.	table[0..15], each slot is a bucket
Bucket index formula	index = (h ^ h>>>16) & (n-1) where h = key.hashCode(), n = table.length	Bitwise AND requires n = power of 2
Load factor 0.75	Resize triggered when size > capacity * 0.75; doubles capacity on resize	16 buckets -> resize at 13th entry
Treeification	Bucket with >= 8 nodes -> red-black tree; <= 6 nodes -> revert to list	O(n) list -> O(log n) tree
hashCode contract	equals()==true REQUIRES same hashCode. Collisions (same hash, not equal) are OK	If a.equals(b) then a.hashCode()==b.hashCode()
hashCode design	Use Objects.hash(field1, field2...) with EXACTLY the same fields as equals()	Objects.hash(id) if equals uses only id
equals checklist	1.this==obj check 2.null check 3.instanceof/getClass 4.cast 5.field comparison	Objects.equals() for null-safe field compare
Mutable key danger	Mutating a key after insertion changes its hashCode -> entry unreachable forever	Use String, Integer, UUID, Records as keys
Null key	HashMap: one null key in bucket 0. TreeMap/ConcurrentHashMap: no null keys	ConcurrentHashMap: null = ambiguity risk
HashMap vs Hashtable	Hashtable: synchronized, no null. HashMap: not synchronized, allows null	Use ConcurrentHashMap for thread safety
LinkedHashMap	HashMap + doubly-linked list = insertion order OR access order preserved	LRU cache: accessOrder=true + removeEldestEntry
TreeMap	Red-black tree, sorted keys, O(log n). NavigableMap: floor/ceiling/headMap/tailMap	Scheduler: TreeMap
ConcurrentHashMap	CAS on empty buckets, synchronized per-bin. No full-table lock. No nulls.	High-concurrency read/write maps
EnumMap	Array-backed, indexed by enum ordinal. O(1), fastest for enum keys.	Map -> EnumMap
WeakHashMap	Keys stored as WeakReferences; GC can collect keys; entry auto-removed	Cache where key lifecycle is managed externally
Map.of()	Immutable, up to 10 entries, no null, unspecified order. Throws on mutation	Config maps, test fixtures
computeIfAbsent	Creates value lazily only if key absent; atomic in ConcurrentHashMap	Nested map init, cache population
entrySet() iteration	Preferred: O(1) access to both key and value. Avoid keySet()+get() (double lookup)	map.forEach((k,v)->...) is equivalent